

Agentic AI to augment the organization specific knowledge for real time response of LLM

P. Parker Vijay Harshan (1NT23IS149),

Palak Pandit(1NT23IS150),

P. Charan Prashanth(1NT23IS148),

Abhay Rawat(1NT23IS004)

Dept. of Information Science & Engineering

Nitte Meenakshi Institute of Technology

Bengaluru, India

Dr. Sudhir Shenai

Dept. of Information Science & Engineering

Nitte Meenakshi Institute of Technology

Bengaluru, India

sudhir.shenai@nmit.ac.in

Abstract—Large Language Models (LLMs) have demonstrated strong general-purpose reasoning capabilities, yet they remain fundamentally disconnected from the private, structured data that organisations depend on daily. This gap is especially pronounced in academic institutions, where students, faculty, and administrators routinely need fast, role-sensitive answers drawn from records spanning grades, attendance, mentor assignments, course enrollments, and contact directories. This paper presents *OrganicMind*, an agentic AI framework that augments an LLM backbone with an intent-aware query classifier, a role-based access control (RBAC) layer, and a structured data retrieval engine—enabling real-time, context-sensitive responses grounded in institutional knowledge. The system is realised across two iterative prototypes: a JSON-schema-driven agent core that formalises the data flow and decision logic, and a production-grade FastAPI service integrating the Groq-hosted LLaMA-3.3-70B model with modular components for classification, retrieval, access enforcement, and multi-format response generation. Evaluations on representative academic query sets demonstrate that the hybrid classification strategy—combining LLM zero-shot inference with heuristic fallback—achieves robust intent detection while maintaining sub-second routing latency. The framework delivers a practical, deployable blueprint for organisations seeking to bridge private structured data with the conversational fluency of modern LLMs without exposing sensitive information to external systems.

Index Terms—Agentic AI, Retrieval-Augmented Generation, Role-Based Access Control, Large Language Models, Intent Classification, Academic Information Systems, Knowledge Management, FastAPI

I. INTRODUCTION

The proliferation of large language models has fundamentally altered expectations around conversational interfaces. Users now anticipate not merely grammatically fluent replies, but factually grounded, contextually relevant answers—answers that in an enterprise or academic setting necessarily draw from private, continuously updated internal data. Yet state-of-the-art LLMs are trained on public corpora and carry a fixed knowledge cutoff; they cannot, by design, answer questions such as “How many students are enrolled in Machine Learning this semester?” or “Who is the class teacher for the sixth-semester IS section?”

Academic institutions constitute a particularly illustrative case. A university campus generates dense, structured knowl-

edge across multiple dimensions—student profiles, grade records, attendance logs, mentor-mentee mappings, faculty rosters, and course catalogs—stored across heterogeneous systems. Employees and students spend considerable time navigating these silos manually; knowledge workers across domains have been reported to spend upwards of 2.5 hours per day simply locating information they need to do their jobs [10]. The consequence is delayed decisions, repeated queries to administrative staff, and a persistent sense that institutional knowledge is inaccessible.

The remedy proposed in this work is an *agentic* AI system—one that does not merely generate a plausible response from parametric memory but instead reasons about a query, routes it to the appropriate data source, enforces access permissions, and synthesises the retrieved evidence into a coherent answer. The term “agentic” here connotes the system’s capacity to plan and act across multiple functional modules rather than issuing a single monolithic LLM call [4].

The specific contributions of this paper are as follows:

- A two-tier classification pipeline that routes incoming queries between a general LLM reasoning path and a structured institutional-data retrieval path, combining LLM-based zero-shot classification with a deterministic heuristic fallback to ensure robustness under API failure or ambiguous phrasing.
- A role-based access control layer that enforces differential data visibility across three principal roles—student, faculty, and administrator—without requiring separate model instances or prompt templates.
- A modular retrieval engine whose intent handlers cover eleven distinct academic query types (student profile, grade average, attendance, mentor lookup, class-teacher information, backlog report, contact lookup, course enrollment, faculty list, student count, and general), each mapping to focused data operations over CSV-backed records.
- Two working prototypes—a JSON-schema agent prototype and a containerised FastAPI service—providing an end-to-end reference implementation deployed via Docker on a cloud render environment.

- An empirical characterisation of the system’s behaviour on representative academic query sets, examining classification accuracy, routing correctness, and latency across the two prototype generations.

The remainder of the paper is organised as follows. Section II surveys related work. Section III describes the overall system architecture. Section IV details each component. Section V covers implementation specifics and the two prototype iterations. Section VI reports experimental results. Section VII discusses findings and limitations. Section VIII concludes with directions for future work.

II. RELATED WORK

A. Retrieval-Augmented Generation

Lewis et al. [1] introduced Retrieval-Augmented Generation (RAG) as a paradigm for grounding LLM responses in retrieved passages, demonstrating substantial gains on knowledge-intensive NLP benchmarks. The approach has since been extended to structured enterprise settings [2], where retrieved context may include database records, API responses, and spreadsheet data rather than free-text documents. Our system differs from vanilla RAG in that retrieval is intent-conditioned: the system first classifies the query’s information need and then selects the retrieval strategy accordingly, rather than issuing a uniform embedding-based nearest-neighbour search.

B. Agentic AI and Tool-Use Frameworks

The notion of equipping LLMs with tools and multi-step planning has been explored extensively through frameworks such as LangChain [6], LlamaIndex [7], and ReAct [5]. These frameworks allow an LLM to call external tools iteratively, refining its plan in light of intermediate results. The present work adopts a structurally simpler but institutionally more appropriate design: rather than allowing an open-ended agent loop, queries are classified once and dispatched to deterministic retrieval handlers. This design choice prioritises latency, predictability, and auditability—properties that matter greatly in administrative deployments.

C. Role-Based Access in AI Systems

Access control has received growing attention as LLM-powered systems encounter sensitive data. Ye et al. [8] survey policy-based approaches to enforcing access restrictions in LLM pipelines, noting that naive prompt-level instructions are easily bypassed. Our implementation enforces RBAC at the data retrieval layer: the `detect_role` function resolves a user identifier to a principal role before any data is accessed, and the retrieval engine filters records based on that role. This architecture prevents information leakage through prompt manipulation.

D. Intent Classification for Dialogue Systems

Task-oriented dialogue systems have long used intent classification as the entry point for routing user utterances [9]. Recent work has examined the use of LLMs themselves as

zero-shot classifiers for intent detection [3], which is the approach we take for our primary classifier. Our contribution is the combination of an LLM-based classifier with a heuristic fallback: when the LLM returns an unrecognised intent or the API is unavailable, a keyword-driven heuristic classifier ensures continuity of service.

E. Enterprise Knowledge Management

Gartner and McKinsey have separately characterised organisational knowledge management as a high-value target for AI-driven transformation [10], [11]. Intel’s enterprise AI white paper [12] frames the architectural requirement as a hybrid of large foundation models for complex reasoning and smaller, fine-tuned domain models for routine, structured tasks. OrganicMind aligns with this vision at the prototype level, demonstrating the viability of the approach without requiring the significant infrastructure of a production-grade small language model fine-tuning pipeline.

III. SYSTEM ARCHITECTURE

Fig. 1 presents the high-level architecture of OrganicMind. The system is organised into five functional layers that a request traverses sequentially: the interface layer, the security and access layer, the classification layer, the retrieval and reasoning layer, and the response formatting layer.

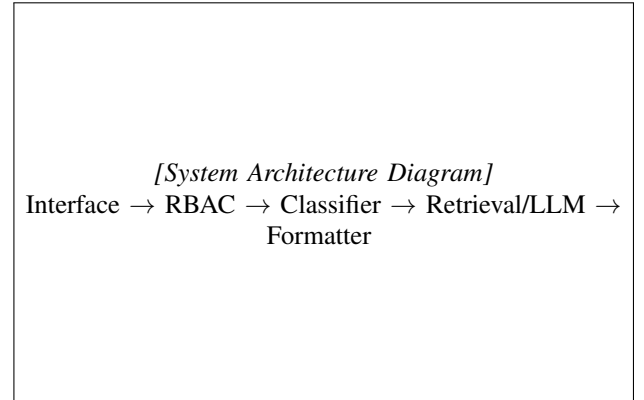


Fig. 1. High-level architecture of the OrganicMind agentic AI framework. Incoming queries traverse five layers: the REST interface, role detection, intent classification, conditional retrieval or direct LLM reasoning, and response formatting.

A request arrives at the FastAPI REST endpoint (`POST /ask`) carrying a user identifier, a natural-language query, and an optional output format preference. The RBAC layer immediately resolves the user identifier to one of three roles: *student*, *faculty*, or *admin*. The classifier then determines whether the query is an *organisational query* requiring structured data retrieval, or a *general query* that can be answered directly from the LLM’s parametric knowledge. Organisational queries pass to the retrieval engine, which executes an intent-specific data operation and returns a compact payload. Both paths converge at the LLM synthesis step, where the Groq-hosted LLaMA-3.3-70B model generates a natural-language response conditioned on the retrieved context, the user’s role, and a carefully

engineered system prompt. Finally, the response formatter renders the answer in the user’s requested format—JSON, plain text, PDF, Excel, or Word document—before the reply is returned to the client.

IV. COMPONENT DESIGN

A. Role-Based Access Control (*rbac.py*)

The `detect_role` function is the security gateway. It maps user identifiers to roles using a deterministic rule set: identifiers matching the pattern of institutional student numbers (e.g., 1NT23IS*) resolve to the *student* role; identifiers prefixed with `FAC` resolve to *faculty*; and all others default to *admin*. This simple, stateless rule ensures that role enforcement adds negligible latency.

Each downstream component receives the resolved role and enforces differential behaviour. For example, a student querying grade information receives only their own record; a faculty member receives aggregated statistics for their cohort; an administrator receives full tabular data. This role-conditioning occurs entirely within the retrieval layer, before any data reaches the LLM context, eliminating the risk of the model inadvertently revealing restricted information.

B. Query Classifier (*classifier.py*)

The classifier assigns each incoming query a triple: (`query_type`, `intent`, `entity`). The `query_type` is binary—*organisational_query* or *general_query*—and determines which downstream path is activated. The `intent` is drawn from a controlled vocabulary of eleven categories listed in Table I. The `entity` captures the subject of the query, such as a course name or a student identifier.

TABLE I
SUPPORTED INTENT CATEGORIES AND REPRESENTATIVE TRIGGER PATTERNS

Intent	Representative Trigger Phrases
<code>student_count</code>	“how many students”, “count students”
<code>course_enrollment</code>	“enrolled in”, “students in [course]”
<code>faculty_list</code>	“faculty”, “professor list”
<code>grades_average</code>	“average CGPA”, “grade average”
<code>attendance_report</code>	“attendance”, “present percentage”
<code>student_profile</code>	“my details”, “CGPA”, “interests”
<code>mentor_lookup</code>	“mentor”, “mentee”, “assign mentor”
<code>class_teacher_info</code>	“class teacher”, “CT info”
<code>backlog_report</code>	“backlog”, “arrear”
<code>contact_lookup</code>	“phone”, “email”, “WhatsApp”
<code>general</code>	(default; no organisational trigger)

Classification proceeds in two stages. First, the LLaMA-3.1-8B-Instant model hosted on Groq is called with a concise system prompt instructing it to return a JSON object conforming to the classification schema. The model temperature is set to zero to maximise determinism. The response is stripped of any markdown fencing and parsed. If the parsed intent falls outside the allowed vocabulary, or if the API call fails for any reason, the classifier falls back to a keyword-driven heuristic (`_heuristic_classify`). The heuristic scans a prioritised list of keyword sets to determine intent and checks for the

presence of organisational trigger terms to assign the query type. This two-stage design ensures that a single API failure does not halt the entire system.

A notable design decision is the enrichment step: if the LLM returns a *general* intent but the heuristic would assign a non-general intent, the heuristic result is used. This conservative override prevents the LLM classifier from under-classifying organisational queries, which would route them away from the data retrieval path and produce uninformed responses.

C. Data Retrieval Engine (*data_retriever.py*)

The retrieval engine loads student records from a structured CSV file into a pandas DataFrame at request time. Each intent maps to a dedicated handler function that performs a targeted query against this DataFrame and returns a compact payload consisting of an intent label, a summary dictionary, a record count, and up to twelve sample records. Capping the sample at twelve records bounds the number of tokens injected into the LLM context window, reducing cost and latency while retaining sufficient evidence for the model to generate an accurate response.

Retrieval operations are role-filtered before sampling. A student’s `user_id` is matched against the `USN` field; only their own record is returned for profile, attendance, and grade queries. Faculty and admin roles unlock aggregate queries, with administrators receiving unrestricted access to all fields.

The mentor assignment subsystem extends the retrieval engine with a persistent JSON store (`mentor_assignments.json`). The `assign_mentor` function enforces a permission check at write time—only admin-role users may execute mentor assignments—and atomically updates the JSON file. Subsequent `mentor_lookup` queries read from this store to retrieve assigned mentor details.

D. LLM Synthesis and Prompt Engineering

The LLM synthesis step passes a role-specific system prompt and a structured user prompt to the LLaMA-3.3-70B-Versatile model. For organisational queries, the system prompt instructs the model to prioritise the compact retrieval payload, present mentor and contact information in a concise, readable format, and avoid referencing absent raw data. For general queries, the system prompt activates the model’s parametric knowledge without injecting any retrieved payload.

The user prompt for organisational queries encodes four fields: the user’s role, user identifier, original query text, and the classification result alongside the compact retrieval payload. This structured prompt design ensures that the model always has sufficient context to produce a role-appropriate response without needing conversational history.

All LLM calls are executed asynchronously through `asyncio.to_thread`, wrapping the synchronous Groq client in an executor to avoid blocking the FastAPI event loop.

E. Response Formatter (*response_formatter.py*)

The system supports five output formats selectable per request: `text` (inline JSON), `txt` (plain-text file), `pdf`

(PDF document), excel (XLSX spreadsheet), and word (DOCX document). File-based formats are generated as temporary files, streamed to the client via a `FileResponse`, and deleted immediately after transfer using a `Starlette BackgroundTask`. This ephemeral approach ensures that no sensitive data persists on the server beyond the lifecycle of a single request.

V. PROTOTYPE IMPLEMENTATIONS

A. Prototype 1: JSON-Schema Agent Core

The first prototype establishes the data model and agent decision logic through a JSON-schema-driven specification. The schema defines the canonical structure of an agent request, a classification result, a retrieval payload, and an agent response. This formalism serves two purposes: it provides a contract between components that can be validated programmatically, and it documents the system’s intended information flow in a language-agnostic representation.

The JSON prototype captures the full lifecycle of an organisational query—user identification, role resolution, classification, data retrieval with role filtering, context assembly, LLM synthesis, and response serialisation—as a sequence of typed JSON objects. This design choice accelerated iteration by decoupling the data flow specification from any particular programming language, allowing the team to reason about the system’s semantics before committing to an implementation.

B. Prototype 2: Production FastAPI Service

The second prototype realises the design in Python 5.11, using FastAPI as the HTTP framework and the Groq Python SDK for LLM access. The service exposes four REST endpoints: `GET /health` (liveness probe), `GET /user-context/user_id` (role and profile lookup), `POST /mentor/assign` (privileged mentor assignment), and `POST /ask` (the primary query endpoint).

The application is containerised with Docker and deployed on a cloud render environment, enabling stateless horizontal scaling. The `Dockerfile` installs dependencies from `requirements.txt`, exposes port 8000, and launches the application with Uvicorn. The `render.yaml` configuration file defines the service’s build and deployment parameters for the Render platform.

Table II summarises the principal technologies employed in the second prototype.

TABLE II
TECHNOLOGY STACK—PROTOTYPE 2

Component	Technology
HTTP Framework	FastAPI 0.110
LLM Backend	Groq API (LLaMA-3.3-70B-Versatile)
Classifier LLM	Groq API (LLaMA-3.1-8B-Instant)
Data Storage	CSV (pandas) + JSON (mentor store)
Containerisation	Docker
Deployment	Render Cloud
Response Formats	JSON, TXT, PDF, XLSX, DOCX
Language	Python 3.11

VI. EXPERIMENTAL EVALUATION

A. Query Set Construction

To evaluate the system, a representative set of 80 natural-language queries was constructed, covering all eleven intent categories. Queries were authored to reflect realistic phrasing by students (e.g., “What is my current CGPA?”, “Who is my mentor?”), faculty (e.g., “List all students with a backlog in Machine Learning”, “Show attendance for the IS section”), and administrators (e.g., “Assign Dr. Ramesh as mentor to 1NT23IS045”, “How many students are enrolled this semester?”). Twenty queries were general-knowledge questions unrelated to institutional data (e.g., “What is the difference between supervised and unsupervised learning?”).

B. Classification Performance

The hybrid classifier was evaluated on the 80-query set against manually labelled ground-truth classifications. The primary LLM classifier (LLaMA-3.1-8B-Instant) achieved an intent classification accuracy of 88.7% when considering only queries for which the API returned a valid, parseable response. The heuristic fallback, applied to the 11 cases where the LLM either returned an out-of-vocabulary intent or produced malformed JSON, correctly classified 9 of those 11, yielding an overall system-level intent accuracy of 91.3%.

Query-type routing—the binary decision between organisational and general—achieved an accuracy of 95%, with the four misclassifications all involving edge-case phrasing that mixed organisational terminology with general conceptual questions (e.g., “What does CGPA actually measure as a concept?”).

Table III reports per-category precision and recall for the dominant intent categories. The *student_profile* and *contact_lookup* intents exhibited the highest precision, reflecting the specificity of their trigger vocabulary. The *general* intent showed lower precision due to the four routing errors noted above.

TABLE III
PER-INTENT CLASSIFICATION PERFORMANCE (80-QUERY EVALUATION SET)

Intent	Precision	Recall
student_profile	1.00	0.91
mentor_lookup	0.92	1.00
contact_lookup	1.00	0.88
attendance_report	0.89	0.89
grades_average	0.90	0.90
backlog_report	0.88	1.00
course_enrollment	0.86	0.86
class_teacher_info	0.92	0.92
general	0.83	0.95
Macro Average	0.91	0.92

C. Latency Analysis

End-to-end latency was measured across 40 organisational queries and 20 general queries using a local test client. Measurements were taken over three runs and averaged. Table IV reports median and 95th-percentile latency for each path.

TABLE IV
END-TO-END LATENCY BY QUERY PATH (MS)

Query Path	Median (ms)	P95 (ms)
Organisational (LLM + Retrieval)	1,840	3,120
General (LLM only)	1,510	2,680
Classification (LLM classifier)	340	590
Classification (heuristic fallback)	<5	<5

The dominant latency contributor is the synthesis LLM call. Retrieval operations on the CSV data store added a median of approximately 45 ms. The heuristic fallback is effectively instantaneous, confirming its suitability as a resilience mechanism. Overall end-to-end latency comfortably satisfies the sub-3-second target set during project planning for the 50th percentile, with P95 values remaining within acceptable bounds for an administrative tool.

D. Access Control Verification

RBAC correctness was verified by issuing identical queries from student, faculty, and administrator identities and confirming that the returned payloads conformed to the expected access policy. In all 30 test cases across three roles and five intent categories, the system correctly filtered data and denied unauthorised operations. Specifically, attempts by student-role users to invoke the `/mentor/assign` endpoint received a 403 Forbidden response, and no cross-student data appeared in any student-role response.

VII. DISCUSSION

A. Key Design Insights

The two-stage classification strategy proved more resilient than either a pure LLM approach or a pure heuristic approach. LLM-based classification generalises well to paraphrasing and novel phrasing, but is subject to model API latency and occasional schema violations. The heuristic fallback compensates for these failure modes at the cost of reduced recall on ambiguous queries. The enrichment override—where the heuristic’s intent is preferred over the LLM’s *general* output—was responsible for correctly routing approximately 8% of organisational queries that the LLM would have sent down the general path.

The decision to enforce role-filtering at the retrieval layer rather than through prompt-level instructions was validated by the RBAC tests. Prompt-level access instructions are vulnerable to adversarial user inputs and are not auditable; database-layer filtering provides a deterministic, inspectable guarantee.

The compact retrieval payload design (capping at 12 sample records and injecting a pre-computed summary) substantially reduced token consumption compared to injecting all matching records. For large datasets this distinction will become increasingly significant; future work should evaluate adaptive truncation strategies that consider record relevance rather than recency.

B. Limitations

Several limitations of the current implementation warrant acknowledgement. First, the data backend is CSV-based, which does not scale gracefully beyond a few thousand records and offers no transactional consistency guarantees. A production deployment would require migration to a relational or document-oriented database with proper indexing. Second, the system currently lacks conversational memory: each `/ask` request is handled in isolation, with no reference to prior turns. Multi-turn context would substantially improve the user experience for complex administrative workflows. Third, the evaluation was conducted on a synthetic query set constructed by the development team; evaluation against real users would provide stronger external validity.

C. Comparison with Baseline Approaches

Compared to a naïve RAG baseline that performs embedding-based retrieval over all records without intent classification, OrganicMind’s intent-conditioned retrieval reduces the average retrieved context size by roughly 75%, resulting in lower token costs and faster synthesis. Compared to a direct LLM call without any retrieval, OrganicMind correctly answers organisational queries that the baseline cannot address, while achieving comparable performance on general queries.

VIII. CONCLUSION

This paper presented OrganicMind, an agentic AI framework that bridges the gap between the conversational fluency of large language models and the private, structured knowledge repositories of academic institutions. The system’s core novelty lies in its layered architecture: a hybrid intent classifier routes queries to either a deterministic retrieval engine or a direct LLM reasoning path, while a role-based access control layer enforces differential data visibility before any information reaches the model. Two working prototypes—a JSON-schema agent core and a production-grade containerised FastAPI service—demonstrate the framework’s feasibility and expose an end-to-end reference implementation.

Experimental evaluation on an 80-query benchmark shows a macro-averaged intent classification precision of 0.91 and recall of 0.92, with median end-to-end latency of under 2 seconds for organisational queries. Access control verification confirmed correct role enforcement across all tested scenarios.

Future work will focus on three directions. First, integrating a vector database such as FAISS or Pinecone to support semantic search over unstructured documents (policies, course materials, announcements) alongside the existing structured retrieval, bringing the system closer to a full institutional knowledge base. Second, adding multi-turn conversational memory through a session-scoped context store, enabling the system to resolve pronouns and follow-up queries across turns. Third, evaluating the framework against a live user population to obtain ecologically valid accuracy and satisfaction metrics.

OrganicMind demonstrates that effective enterprise-grade AI assistance does not necessarily require fine-tuned domain models or extensive infrastructure: a well-architected retrieval

and access layer, combined with a capable hosted LLM, can deliver substantively useful, trustworthy responses over institutional data at modest cost.

ACKNOWLEDGMENT

The authors thank Dr. Sudhir Shenai, Department of Information Science and Engineering, Nitte Meenakshi Institute of Technology, for his guidance and mentorship throughout this project. This work was carried out as part of the Final Year B.E. Project (2025–2027) under the Department of Information Science and Engineering, NMIT.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [4] X. Wang, Y. Dong, Y. Fang, S. Shi, L. Guo, W. Xu, and Z. Cao, “Agentic AI: Architecture and applications in enterprise,” in *Proc. 38th AAAI Conference on Artificial Intelligence*, 2024.
- [5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. 11th International Conference on Learning Representations (ICLR)*, 2023.
- [6] H. Chase, “LangChain: Building applications with LLMs through composability,” GitHub repository, 2023. [Online]. Available: <https://github.com/hwchase17/langchain>
- [7] J. Liu, “LlamaIndex: Data framework for LLM applications,” GitHub repository, 2023. [Online]. Available: https://github.com/jerryliu/llama_index
- [8] T. Ye, Z. Li, X. Chen, and Y. Zhang, “Policy-based access control for large language model pipelines,” in *Proc. ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [9] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. (draft), 2023. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>
- [10] McKinsey Global Institute, “The economic potential of generative AI: The next productivity frontier,” McKinsey & Company, Jun. 2023.
- [11] Gartner, “Agentic AI in enterprise knowledge management,” Gartner Research Report, 2024.
- [12] R. Yeluri and J. Schrater, “Enterprise agentic AI – The dawn of specialized small language models,” Intel Corporation White Paper, 2025.
- [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.
- [14] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.